

Article

Enhancing Cybersecurity: Hybrid Deep Learning Approaches to Smishing Attack Detection

Tanjim Mahmud ^{1,*}, Md. Alif Hossen Prince ¹, Md. Hasan Ali ¹, Mohammad Shahadat Hossain ^{2,3}
and Karl Andersson ³

¹ Department of Computer Science and Engineering, Rangamati Science and Technology University, Rangamati 4500, Bangladesh; alifhossen.prince@gmail.com (M.A.H.P.); mdhasanali.rmstu@gmail.com (M.H.A.)

² Department of Computer Science and Engineering, University of Chittagong, Chittagong 4331, Bangladesh; hossain_ms@cu.ac.bd

³ Cybersecurity Laboratory, Luleå University of Technology, 97187 Luleå, Sweden; karl.andersson@ltu.se

* Correspondence: tanjim.cse@rmstu.ac.bd

Abstract: Smishing attacks, a sophisticated form of cybersecurity threats conducted via Short Message Service (SMS), have escalated in complexity with the widespread adoption of mobile devices, making it increasingly challenging for individuals to distinguish between legitimate and malicious messages. Traditional phishing detection methods, such as feature-based, rule-based, heuristic, and blacklist approaches, have struggled to keep pace with the rapidly evolving tactics employed by attackers. To enhance cybersecurity and address these challenges, this paper proposes a hybrid deep learning approach that combines Bidirectional Gated Recurrent Units (Bi-GRUs) and Convolutional Neural Networks (CNNs), referred to as CNN-Bi-GRU, for the accurate identification and classification of smishing attacks. The SMS Phishing Collection dataset was used, with a preparatory procedure involving the transformation of unstructured text data into numerical representations and the training of Word2Vec on preprocessed text. Experimental results demonstrate that the proposed CNN-Bi-GRU model outperforms existing approaches, achieving an overall highest accuracy of 99.82% in detecting SMS phishing messages. This study provides an empirical analysis of the effectiveness of hybrid deep learning techniques for SMS phishing detection, offering a more precise and efficient solution to enhance cybersecurity in mobile communications.

Keywords: smishing attacks; Short Message Service; phishing; hybrid deep learning



Citation: Mahmud, T.; Prince, M.A.H.; Ali, M.H.; Hossain, M.S.; Andersson, K. Enhancing Cybersecurity: Hybrid Deep Learning Approaches to Smishing Attack Detection. *Systems* **2024**, *12*, 490. <https://doi.org/10.3390/systems12110490>

Academic Editor: Anas M Alsobeh

Received: 15 September 2024

Revised: 8 November 2024

Accepted: 11 November 2024

Published: 14 November 2024



Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

As mobile device usage proliferates, smishing attacks—phishing conducted via Short Message Service (SMS)—have become increasingly sophisticated, complicating individuals' ability to distinguish between legitimate and malicious messages [1]. These attacks exploit social engineering techniques to deceive victims into revealing sensitive information or clicking harmful links, posing significant threats to personal and financial data [2]. The trust users place in SMS communications creates a critical vulnerability in mobile cybersecurity [3–5].

Phishing is a cybercrime technique where attackers impersonate legitimate entities to trick individuals into sharing sensitive information, such as passwords, credit card numbers, or personal details [6]. While phishing is most commonly executed through emails, it can also be conducted through other channels, including phone calls (vishing) and SMS (smishing). Smishing, specifically, involves using SMS text messages to impersonate reputable organizations, such as banks, government agencies, or service providers, to prompt recipients into taking immediate action, like clicking a link or calling a phone number [7]. The messages often convey a sense of urgency, exploiting the immediate and personal nature of SMS to make it harder for users to recognize the scam.

As a subset of phishing, smishing poses unique dangers due to the popularity and trust associated with mobile phones [8]. SMS phishing attacks are often more effective than traditional email phishing because users generally trust text messages more readily [9]. Additionally, attackers leverage social engineering techniques, such as urgency and emotional appeals, to manipulate recipients, increasing the likelihood of a successful attack [10].

Traditional cybersecurity methods for detecting phishing, such as feature-based, rule-based, and heuristic approaches, have struggled to adapt to the rapidly evolving tactics employed by cybercriminals [11]. These conventional techniques often fail to identify novel or subtly altered phishing attempts, leaving users vulnerable to exploitation. The rising frequency and sophistication of smishing attacks highlight the urgent need for advanced detection methods that can respond to emerging threats [12]. According to a 2022 report by Verizon Business, human errors, including phishing and compromised credentials, account for 82% of data breaches [13], while the FBI's Internet Crime Complaint Center reports that phishing was the most prevalent cybersecurity threat in the United States last year, impacting over 323,000 victims [14]. Alarming, only 20% of organizations provide their employees with annual phishing awareness training, exposing a significant gap in cybersecurity preparedness [15]. This landscape underscores the critical need for enhanced detection mechanisms to combat smishing attacks, which can deceptively mimic legitimate messages, complicating detection using traditional methods.

To address these cybersecurity challenges, this study proposes a hybrid deep learning model that combines Bidirectional Gated Recurrent Units (Bi-GRUs) [16] and Convolutional Neural Networks (CNNs) [17], referred to as CNN-Bi-GRU [18]. By leveraging the strengths of both architectures, the model aims to enhance the accuracy of smishing detection, outperforming existing solutions in identifying phishing messages. The SMS Phishing Collection dataset [19] is employed, with unstructured text data converted into numerical representations through preprocessing with Word2Vec [20]. The hybrid model is fine-tuned, achieving an impressive accuracy of 99.82% in detecting smishing messages, thereby demonstrating the potential of deep learning techniques in fortifying cybersecurity defenses.

This study contributes to ongoing efforts to improve cybersecurity and protect users from the growing threat of smishing. By offering a precise and efficient solution for detecting SMS phishing, this research aims to enhance the cybersecurity of smartphone users globally, ultimately reducing the risks associated with these deceptive cyber threats.

The main contributions of this study are as follows:

1. We utilized three distinct SMS datasets, training models individually before merging them into a combined dataset. This approach enhances model generalization and adaptability, as demonstrated by comparative performance analyses through all datasets, providing insights into the models' effectiveness on varied data sources.
2. We investigated the impact of different training parameters on the model's performance and employed the Explainable AI (XAI) method LIME to interpret model decisions, adding transparency to the detection process.

2. Review of Existing Studies

SMS phishing attacks, or smishing, have increased in prevalence due to the increasing use of mobile devices for online communication and commerce. Over the years, several studies and models have been developed to detect and prevent smishing attacks using machine learning algorithms. In 2017, Joo et al. [21] used statistical learning to detect smishing attacks. Moving on to 2020, Mishra and Soni [10] utilized a prototype containing an APK download detector, URL filter, code analyzer, and content analyzer to evaluate SMS and URL behavior for smishing detection.

Recent advancements in SMS phishing detection have employed a variety of methodologies. Gunikhan Sonowal [22] utilized four algorithms and machine learning classifiers to rank 52 SMS properties, while Roy et al. [23] developed a system for classifying SMS spam using deep learning techniques. Ghourabi et al. [24] introduced a CNN-LSTM hybrid model for Arabic and English SMS messaging. Jain [25] proposed a zero-day smishing

detection approach, emphasizing feature prioritization. This year, Xia and Chen [26] developed a discrete hidden Markov model leveraging word order data to address low-term frequency challenges in SMS spam detection. In 2021, Mishra and Soni [27] achieved a 5-CV accuracy of 97.93%, while Liu et al. [28] compared a modified spam Transformer to nine traditional methods for SMS spam recognition. Finally, Mishra and Soni [29] identified smishing SMS attacks, and Mambina et al. [30] proposed a mixed machine learning approach for Swahili smishing detection. Table 1 presents summary of related works.

Table 1. Summary of the related works regarding SMS phishing detection.

Authors	Period	Method	Accuracy	Observations
Joo et al. [21]	2017	Statistical learning method	-	No deep learning models
Roy et al. [23]	2020	CNN-LSTM	99.44%	English texts only
Ghourabiet al. [24]	2020	CNN-LSTM	98.37%	No URL or file analysis.
Mishra and Soni [10]	2020	Structured content analysis	96.12%	Few effective options to prevent threats, and APK malware detection is difficult.
Gunikhan Sonowal [22]	2020	Pearson, Spearman, Kendall, and Point-biserial correlations.	98.40%	Extensive analysis for feature selection
Ankit Kumar Jain [25]	2020	Classifier implementation and Information gain values	96%.	English texts only. No URL analysis. Minimal dataset size.
Xia and Chen [26]	2020	Discrete hidden Markov model	98.5%	No word labeling. No HMM versions were tested.
Mishra and Soni [27]	2021	Backpropagation Algorithm	97.93%	A signature is difficult to generate.
Liu et al. [28]	2021	Modified model based on the vanilla Transformer	98.92%	Minimal dataset size.
Mishra and Soni [29]	2022	SMS service with custom rules.	97.40%	No deep learning SMS Phishing detection models.
Mambina et al. [30]	2022	TFIDF and feature selection in extra tree classifiers	99.86%	No DL models.
Baardsen et al. [31]	2022	BERT Embedding	97.9%	Used Email Dataset.

Using machine learning methods to detect smishing attacks is a viable strategy for limiting the associated dangers. Current machine learning algorithms mainly rely on feature engineering, which may be time-consuming and not always able to keep up with their dynamic nature. Numerous studies analyzed in this investigation demonstrate that machine learning and deep learning algorithms accurately identify smishing messages. Further study is required to develop algorithms to detect increasingly sophisticated at-

tacks. An analysis has been conducted on the limitations and prospective advancements of various spam and smishing detection systems. Some constraints include language dependence, insufficient detection accuracy, and short training sets. We must determine how deep learning algorithms can identify SMS phishing attempts to defend against these complicated and potentially catastrophic attacks. We analyze a public dataset [32] of SMS phishing attacks utilized in evaluating the proposed deep learning method with adequate accuracy due to meticulous preprocessing procedures and changing hyperparameters with cross-validations for successful models and future research into identifying SMS phishing attacks. Formal training, testing, and validation sets are created from the data set. In the following section, we discuss the methodology employed in this work to accomplish the intended experiments.

3. Study Design and Implementation

Developing an SMS phishing detection system requires consideration of several factors. It is essential to load and configure the training data initially. The text data must be preprocessed to ensure training accuracy by removing whitespace, punctuation, and stop words. A Word2Vec model can organize and clean the data before generating an embedding matrix for the network.

3.1. Datasets

The first step in building a robust classification model for SMS phishing (smishing) attack detection involves gathering a comprehensive dataset. For this study, we utilized three key datasets to ensure model accuracy through different SMS phishing detection scenarios.

1. **SMS Phishing Collection:** The primary dataset used in this study is the SMS Phishing Collection, curated by Almeida et al. [19], which is publicly available on Kaggle [32]. This dataset contains a total of 5571 SMS messages, comprising 4824 legitimate (ham) messages and 747 phishing (smishing) attempts (see Table 2).
2. **Phishing_detection dataset:** For further testing, we used a second dataset with a total of 13,320 samples, which includes 7981 spam and 5339 legitimate (ham) messages (https://huggingface.co/datasets/Ad10sKun/phishing_detection, accessed on 12 September 2024). This dataset broadens the coverage of potential spam scenarios and aids in evaluating the model's performance on both spam and legitimate messages.
3. **Phishing-dataset Dataset:** Lastly, we leveraged a third dataset (<https://huggingface.co/datasets/ealvaradob/phishing-dataset>, accessed on 12 September 2024) consisting of 5971 SMS messages that were originally categorized as ham, smishing, and spam [10,33]. To align with the binary classification objective of this study, we restructured this dataset into two categories: legitimate (ham) messages totaling 4844 and smishing messages totaling 1127.
4. **Combined Dataset:** We created a fourth combined dataset by merging the SMS Phishing Collection, the Phishing_detection Dataset, and the Phishing-dataset. This new dataset aggregates all samples to enhance the robustness of our analysis, leading to a total of 24,862 samples, including 15,007 legitimate (ham) messages and 9855 phishing (smishing) attempts.

Using these three datasets allows us to thoroughly evaluate our hybrid deep learning model's effectiveness in detecting smishing attacks and distinguishing them from other SMS types.

Table 2. Distribution of Ham and Smishing Samples Across Datasets.

Dataset	Ham (Legitimate)	Smishing (Phishing)	Ham to Smishing Ratio
SMS Phishing Collection (Dataset 1)	4824	747	6.46:1
Phishing_detection dataset (Dataset 2)	5339	7981	1:1.5
Phishing-dataset (Dataset 3)	4844	1127	4.3:1
Combined Dataset (Dataset 4)	15,007	9855	1.52:1

3.2. Data Preprocessing

Before entering raw SMS message data into the deep learning model, preprocessing is necessary. The input text must be sanitized by removing unnecessary information, such as stop words, punctuation, and special characters [34]. The widely used Word2Vec method converts text into numerical vectors by representing each word in a multi-dimensional vector space, where each word can influence the context of other words in the text. This enables the deep learning model to parse text input, recognize patterns in numerical vectors, and make accurate predictions.

3.2.1. Text Cleanup

Deep learning algorithms require SMS preprocessing, which is essential [35]. During this process, filler words such as 'the', 'and', 'or', and 'a', which do not significantly contribute to the text's meaning, are removed. This enhances the model's efficiency by cleaning the data. Consistency is also improved by converting all text to lowercase. Depending on the specific purpose and dataset, additional techniques may be applied, such as stemming, lemmatization, and handling rare or misspelled words. Text preprocessing aims to standardize and cleanse raw text input, enabling deep learning algorithms to make accurate predictions.

3.2.2. Process of Tokenization

Tokenization breaks down a written document into smaller parts, typically single words or phrases [36–38]. In SMS text processing, tokenization refers to the practice of segmenting raw text data into individual words or tokens. Since machine learning models require numerical input, tokenization is a crucial step in text processing. The Python Natural Language Toolkit (NLTK) package supports text data tokenization. By using the `tokenize()` function, input text is split into individual words, resulting in a list of these words. Various tokenization approaches and criteria can be customized to fit a specific purpose or dataset. Text data can be naturally divided into tokens based on whitespace, punctuation, and individual letters. Using regular expressions and other techniques, custom tokenization rules can also be created to meet unique requirements.

3.2.3. Padding

After SMS messages have been tokenized, the resulting sequences need to be padded to a uniform length, as machine learning algorithms require inputs of the same size [39]. Therefore, all inputs must contain the same number of features. This process, known as padding, typically involves adding zeros at the beginning or end of sequences. For example, consider two tokenized phrases: 'I prefer tea' and 'I hate tea and coffee'. The first sentence contains three tokens, while the second has five. To make their lengths equal, we can pad the first sentence with two zeros at the end, resulting in 'I prefer tea 0 0', and add a single zero at the beginning of the second sentence to make it '0 I hate tea and coffee'.

3.2.4. Word Embedding

Semantic research utilizes word embeddings, also known as vector representations, to represent words in a meaningful way. Word2Vec is commonly used to generate these word embeddings by predicting word contexts within a corpus. For example, given words like 'prefer', 'tea', and 'and', the Word2Vec algorithm might predict the phrase 'I prefer tea and coffee', based on these context terms.

3.2.5. Embedding Matrix

The learned word embeddings are fed into the neural network, which produces an embedding matrix [40]. The structure of the embedding matrix is based on the vocabulary size (total unique words in the dataset) and the embedding size. This matrix has dimensions of (vocab size, embedding size), with each row representing a word from the dataset. Each cell in the matrix contains numerical values that represent the word embeddings. The

embedding matrix can be used to initialize the weights of the neural network's embedding layer. For example, if our vocabulary contains 4203 terms and the embedding size in our Word2Vec model is 100, the embedding matrix dimensions would be 4203 by 100, or (4203, 100).

3.3. Proposed Model Architecture

After preprocessing the SMS dataset and constructing an embedding matrix, a CNN-Bi-GRU hybrid model is trained to recognize SMS phishing. This proposed model comprises three main components: an embedding layer, a CNN layer, and a Bi-GRU layer. The embedding layer extracts syntactic and semantic information from the text and converts it into a fixed-length vector representation. The CNN layer applies multiple convolutional filters to the word embeddings, generating feature maps through convolutions and nonlinear activation functions. These feature maps are then integrated, normalized, and passed to the Bi-GRU layer, with max pooling preserving distinctive features while reducing spatial dimensions. The Bi-GRU layer combines outputs from both GRU units, which are then passed through a dense (fully connected) layer that applies a nonlinear activation function to produce the final output. This collaborative layering enables meaningful representations from text data, capturing pertinent information for phishing detection. The methodology followed in this study is illustrated in Figure 1.

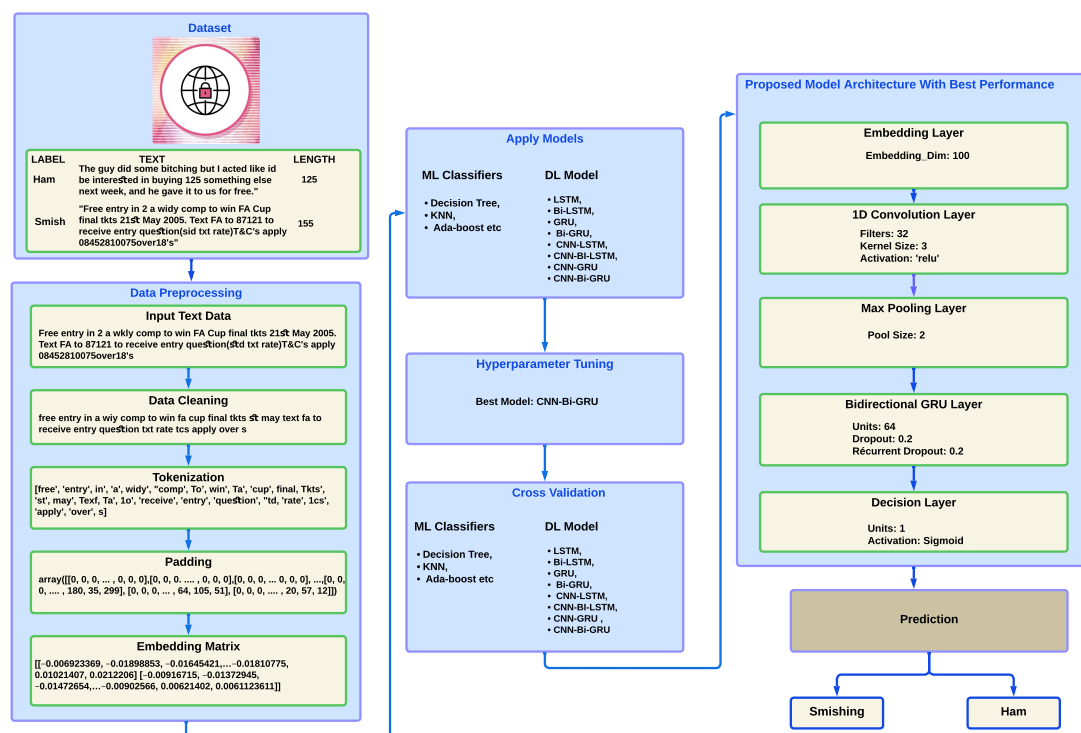


Figure 1. Proposed system architecture.

For model compilation, the log loss function and the Adam optimizer are used, and a randomized search method is employed to fine-tune hyperparameters such as epochs, batch size, dropout rate, and CNN layer filters. The model's performance is evaluated on a distinct test set, while a validation set helps identify optimal hyperparameters, assessing metrics like precision, recall, accuracy, and F1-score. Built with the Keras Sequential API, the architecture includes a 1D convolutional layer with 32 filters (kernel size of 3), an embedding layer, a max pooling layer, a dense output layer with a sigmoid activation function, and a bidirectional GRU layer with 64 units. Once the Word2Vec embeddings are generated, the model is trained iteratively on the available data, splitting them into training and validation sets. The model is trained in batches of size 128 for 50 epochs with binary

cross-entropy loss and the Adam optimizer, iteratively achieving the desired accuracy level through the validation data.

4. Experimental Outcomes and Analysis

In this section, we present a thorough empirical investigation carried out in this work for detecting SMS phishing attacks.

4.1. Setup

In this study, we utilized a Tesla T4 GPU, paired with an Intel® Xeon® CPU operating at 2.00 GHz, to implement our proposed approaches for smishing attack detection (see Table 3). The Tesla T4 is a powerful GPU specifically designed for machine learning and deep learning tasks, featuring NVIDIA's Tensor Cores, which significantly accelerate training and inference operations. This hardware setup allows for the efficient processing of large datasets and complex models, which is critical in the context of cybersecurity where timely and accurate detection of smishing attacks is paramount. The total memory capacity of 13,290,460 kB (approximately 13 GB) facilitates the handling of substantial datasets and complex feature sets. This memory capacity supports the training of deep learning models without the risk of memory overflow, thus enabling more extensive experiments and model configurations. The effective utilization of the NVIDIA-SMI 535.104.05 driver ensures optimized performance and stability during model training and evaluation phases.

Table 3. Device specifications.

Component	Specifications
Device Name	Tesla T4
Model Name	Intel® Xeon® CPU @ 2.00 GHz
Total Memory	13,290,460 kB
GPU Driver Version	NVIDIA-SMI 535.104.05

4.2. Dataset Splitting

To ensure the efficacy and precision of a machine learning model for SMS phishing detection, dividing the dataset into training, validation, and testing groups is essential. Often, a split ratio of 75%:10%:15% is employed for this purpose. Stop words must be deleted during preprocessing for the machine learning model to learn from text data more effectively. When the dataset has been preprocessed, it can be divided randomly into three groups. Ensuring each collection has a representative mix of legitimate and smishing messages is essential, although the division can be arbitrary. The validation set is regularly used during learning to assess the model's accuracy, ensure its development, and prevent overfitting [41], while the training set is utilized to teach the framework about its functionality. Using unlabeled data, the testing set assesses the model's performance and determines its quality. They can be trained, validated, and tested using this method to properly recognize SMS phishing messages, safeguarding individuals and companies from cyberattacks. Because they can learn features from raw data, deep learning models are more successful than conventional machine learning techniques for recognizing SMS phishing communications. Statistics of the Smishing Collection Dataset [19,32] are given below in Table 4.

Table 4. Dataset statistics (Dataset 1).

Message Type	Number of Messages	Training Size	Testing Size	Validation Size
Smishing	746 (13.4%)	559 (75%)	112 (13.4%)	75 (13.5%)
Ham	4824 (86.6%)	3618 (25%)	726 (86.6%)	480 (86.5%)
Total	5570	4177 (75%)	838 (15%)	555 (10%)

4.3. Model Evaluation Results

Models include decision tree, random forest, KNN, SVM, Ada-Boost, LSTM, Bi-LSTM, GRU, CNN-LSTM, CNN-Bi-GRU, CNN-GRU, and CNN-Bi-LSTM for an experiment. The CNN-Bi-GRU model performed well, with an F1 score of 98.56 and an accuracy of 98.54. Precision scores of 0.9939 and recall scores of 0.9775 were above average compared to the other models investigated. Initial deep learning model hyperparameters were as follows: dropout rate 0.2, filters 32, units 64, and recurrent rate 0.2 for deep experimented deep learning models. On the output, the sigmoid activation is used, whereas the ReLU activation is used on the input. Each model was applied to join batch 128, epoch 50. The CNN-Bi-GRU model was highly influential in identifying SMS phishing attempts. In addition, we achieved satisfactory results through all performance metrics [42]. The evaluation findings for the SMS phishing detection dataset 1 are summarized in Table 5.

Table 5. Comparison of algorithm performance on Dataset 1.

Algorithm	Accuracy	Precision	Recall	F1-Score	ROC AUC	AP
DECISION TREE	0.9747	0.9568	0.996	0.9758	0.9741	0.955
RANDOM FOREST	0.9731	0.9731	0.975	0.9738	0.9731	0.961
KNN	0.9309	0.899	0.975	0.9353	0.9298	0.889
SVM	0.8871	0.9127	0.862	0.8867	0.8877	0.858
ADA-BOOST	0.9209	0.9147	0.933	0.9235	0.9206	0.888
LSTM	0.9708	0.9687	0.975	0.9716	0.9947	0.996
BI-LSTM	0.9724	0.9759	0.970	0.9729	0.9958	0.997
GRU	0.9754	0.9746	0.978	0.976	0.9958	0.996
BI-GRU	0.9708	0.9922	0.951	0.9709	0.9957	0.996
CNN-LSTM	0.9724	0.9817	0.964	0.9728	0.9971	0.998
CNN-BI-LSTM	0.9731	0.9702	0.978	0.9739	0.9975	0.998
CNN-GRU	0.9693	0.9617	0.979	0.9703	0.9975	0.998
CNN-BI-GRU	0.9854	0.9939	0.978	0.9856	0.9986	0.999

On the other hand, the performance analysis of various algorithms through Datasets 2, 3, and 4 demonstrates that deep learning models consistently outperformed traditional machine learning algorithms in detecting SMS phishing, especially hybrid models. For Dataset 2, CNN-BiGRU achieved the highest accuracy at 97.22% (see Table 6), while Dataset 3 saw the same model lead again with 98.17% accuracy (see Table 7). Notably, Dataset 4, a combined dataset comprising Dataset 1, Dataset 2, and Dataset 3, reinforced CNN-BiGRU's effectiveness, reaching an accuracy of 98.21% with high precision, recall, and F1 scores, showcasing its adaptability to various diverse datasets (see Table 8). Overall, CNN-BiGRU consistently delivered the most robust results, demonstrating superior generalizability and high AUC-ROC scores. These findings highlight the value of hybrid deep learning approaches, particularly CNN-BiGRU, for versatile and accurate smishing detection through various data contexts.

Table 6. Performance comparison of models on test Dataset 2.

Algorithm	Accuracy	Precision	Recall	F1 Score	AUC-ROC Score
Decision Tree	0.8482	0.8706	0.8746	0.8726	0.8420
Random Forest	0.7386	0.8875	0.6417	0.7448	0.7612
K-Nearest Neighbors	0.6941	0.7387	0.7512	0.7449	0.6808
SVM	0.6769	0.6640	0.9242	0.7728	0.6192
AdaBoost	0.8126	0.7842	0.9448	0.8570	0.7817
LSTM	0.9638	0.9808	0.9579	0.9692	0.9948
BiLSTM	0.9605	0.9798	0.9532	0.9663	0.9936
GRU	0.9627	0.9808	0.9560	0.9683	0.9935
BiGRU	0.9633	0.9855	0.9523	0.9686	0.9948
CNN-LSTM	0.9583	0.9761	0.9532	0.9645	0.9929
CNN-BiLSTM	0.9588	0.9779	0.9523	0.9649	0.9925
CNN-GRU	0.9583	0.9807	0.9486	0.9643	0.9934
CNN-BiGRU	0.9722	0.9703	0.9486	0.9593	0.9929

Table 7. Performance comparison of models on test Dataset 3.

Algorithm	Accuracy	Precision	Recall	F1 Score	AUC-ROC Score
Decision Tree	0.9564	0.9347	0.9817	0.9576	0.9563
Random Forest	0.9564	0.9658	0.9466	0.9561	0.9565
K-Nearest Neighbors	0.8914	0.8482	0.9543	0.8981	0.8912
SVM	0.8372	0.8234	0.8598	0.8412	0.8371
AdaBoost	0.9037	0.8909	0.9207	0.9055	0.9036
LSTM	0.9648	0.9485	0.9832	0.9656	0.9950
BiLSTM	0.9717	0.9654	0.9787	0.9720	0.9961
GRU	0.9694	0.9625	0.9771	0.9697	0.9961
BiGRU	0.9648	0.9707	0.9588	0.9647	0.9953
CNN-LSTM	0.9702	0.9570	0.9848	0.9707	0.9962
CNN-BiLSTM	0.9732	0.9600	0.9878	0.9737	0.9970
CNN-GRU	0.9763	0.9771	0.9756	0.9764	0.9972
CNN-BiGRU	0.9817	0.9813	0.9619	0.9715	0.9975

Table 8. Performance metrics of different algorithms on Dataset 4.

Algorithm	Accuracy	Precision	Recall	F1 Score	AUC-ROC Score
Decision Tree	0.9245	0.9159	0.9349	0.9253	0.9245
Random Forest	0.8411	0.9051	0.7622	0.8275	0.8411
K-Nearest Neighbors	0.7855	0.7744	0.8061	0.7899	0.7855
SVM	0.7182	0.7586	0.6404	0.6945	0.7182
AdaBoost	0.8080	0.7571	0.9073	0.8254	0.8079
LSTM	0.9768	0.9865	0.9671	0.9767	0.9965
BiLSTM	0.9763	0.9783	0.9744	0.9764	0.9974
GRU	0.9738	0.9824	0.9650	0.9736	0.9958
BiGRU	0.9763	0.9884	0.9640	0.9760	0.9974
CNN-LSTM	0.9724	0.9848	0.9595	0.9720	0.9951
CNN-BiLSTM	0.9721	0.9761	0.9679	0.9720	0.9948
CNN-BiGRU	0.9821	0.9597	0.9857	0.9725	0.9950

4.4. Proposed Model Hyperparameters Tuning

The best values of hyperparameters were used in implementing the system and utilizing hyperparameters of the CNN-Bi-GRU model. The model's performance was carefully calibrated by setting these parameters, and after extensive testing, the optimal values were found. The filter parameter was initialized to 32 and selected from 32, 64, and 128, with 32 representing the best value. The dropout rate was established at 0.2, with 0.3 and 0.4 also considered.

The model was trained through fifty epochs, with possibilities of between fifty and one hundred epochs considered. The Adam optimizer was chosen above the similarly evaluated RMSProp optimizer. The kernel size was initially set to 3, with 5 also being considered; nevertheless, 3 produced superior results. The units parameter was set to 64 from the available 16, 32, 64, and 128. Other viable values of 0.3 and 0.5 were investigated, along with the recurrent dropout value of 0.2. In addition to tanh and sigmoid, the Relu function was utilized for input activation. Sigmoid was selected as the output activation, while Relu and Tanh were also evaluated. Ultimately, a pool size of 2 was used, but 3 was also considered. We are confident that these hyperparameters create the ideal proposed model configuration due to considerable experimentation. A graph illustrating the accuracy of the model for the parameters is shown in Figure 2.

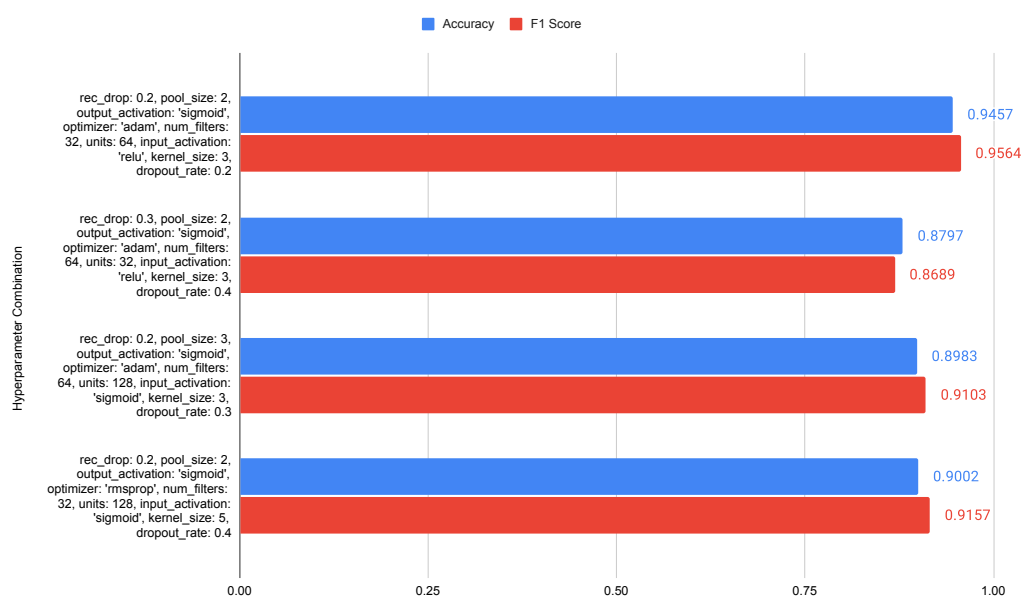


Figure 2. A graph illustrating the proposed model's performance across different parameter sets on Dataset 1.

4.5. Cross-Validation

To address overfitting prevention, we implemented regularization techniques through our models. Dropout layers were added to recurrent layers (LSTM, GRU, and Bidirectional variants) with a 0.2 dropout rate and recurrent dropout. For convolutional models, dropout layers were combined with max-pooling to further mitigate overfitting. Additionally, kernel regularization with l2 (0.01) was applied to the convolutional and recurrent layers, limiting model complexity for better generalization. We also used cross-validation with varied parameter settings to fine-tune performance and validate the model through four datasets.

Table 9 compares the efficacy of three unique cross-validation processes (three-, five-, and ten-fold) on various machine learning algorithms and deep learning models. Model quality is evaluated by predicting a classification problem's outcome. Models include decision trees, random forests, KNN, SVM, Ada-Boost, CNN-Bi-GRU, GRU, CNN-GRU, CNN-LSTM, LSTM, CNN-Bi-LSTM, and Bi-LSTM. Traditional machine learning (ML) approaches are less accurate than deep learning models, with the CNN-Bi-GRU model doing the best through all three cross-validation strategies. With batch sizes of 128 in 3-, 5-, and 10-fold cross-validation processes, respectively, the accuracy was 0.9837, 0.9890, and 0.9974. Overall, the results illustrate how beneficial deep learning models can be for tackling classification problems and how vital it is to use cross-validation to assess model performance.

Table 9. Applied models' results for various folds on Dataset 1.

Model	3-Fold	5-Fold	10-Fold
Decision Tree	0.9574	0.9630	0.9671
Random Forest	0.9509	0.9635	0.9692
KNN	0.8844	0.8940	0.8980
SVM	0.8559	0.8632	0.8696
Ada Boost	0.9182	0.9126	0.9140
LSTM	0.9733	0.9837	0.9917
Bi-LSTM	0.9721	0.9848	0.9910
GRU	0.9689	0.9830	0.9932
Bi-GRU	0.9710	0.9827	0.9913
CNN-LSTM	0.9769	0.9844	0.9911
CNN-Bi-LSTM	0.9710	0.9796	0.9926
CNN-GRU	0.9772	0.9851	0.9935
CNN-Bi-GRU	0.9837	0.9890	0.9974

It appears that the CNN-Bi-GRU model can identify SMS phishing attempts. In actuality, the CNN-Bi-GRU approach operates admirably. The findings have implications for weak companies. Deep learning models can increase SMS phishing security.

In Table 10, we summarize the performance of our proposed model and show the adequate accuracy on the best value from hyperparameter tuning and batch size 32. The same epoch and 10-fold cross-validation were applied to obtain an optimal result. In our study, various parameters significantly influenced the model's performance in smishing detection:

1. Units: Increasing the units in layers enhances the model's capacity to learn complex patterns. For instance, transitioning from 16 to 64 units improved both accuracy and F1 scores, indicating better feature extraction.
2. Batch Size: Smaller batch sizes (e.g., 32) stabilize gradient updates, resulting in improved convergence. This is reflected in the slight gains in F1 scores, suggesting enhanced balance between precision and recall.
3. Kernel and Pool Size: A kernel size of 3 and pooling size of 2 efficiently reduce feature map dimensionality while retaining critical data features, contributing to high performance.
4. Activation Functions: Using ReLU for hidden layers facilitates modeling nonlinear relationships, while Sigmoid in the output layer aids in binary classification, directly impacting the accuracy of predictions.
5. Epochs: Training for 50 epochs strikes a balance between learning and overfitting. Coupled with dropout layers (rate of 0.2), it promotes generalization on unseen data.
6. Filters: The selection of 32 filters captures diverse features from the text efficiently. This ensures computational efficiency without compromising accuracy.

Table 10. Performance of the proposed model across various folds with different parameter sets on Dataset 1.

CV Fold	Units	Batch	Kernel Size	Pool Size	Input Activation	Output Activation	Epoch	Filters	Accuracy	F1 Score
0	16	128	3	2	Relu	Sigmoid	50	32	0.9808	0.9754
3	16	128	3	2	Relu	Sigmoid	50	32	0.9759	0.9823
5	16	128	3	2	Relu	Sigmoid	50	32	0.9863	0.9689
10	16	128	3	2	Relu	Sigmoid	50	32	0.9926	0.9865
0	32	128	3	2	Relu	Sigmoid	50	32	0.9693	0.9579
3	32	128	3	2	Relu	Sigmoid	50	32	0.9752	0.9671
5	32	128	3	2	Relu	Sigmoid	50	32	0.9859	0.9912
10	32	128	3	2	Relu	Sigmoid	50	32	0.9936	0.9876
0	64	128	3	2	Relu	Sigmoid	50	32	0.9808	0.9879
3	64	128	3	2	Relu	Sigmoid	50	32	0.9837	0.9932
5	64	128	3	2	Relu	Sigmoid	50	32	0.9890	0.9971
10	64	128	3	2	Relu	Sigmoid	50	32	0.9974	0.9892
0	64	32	3	2	Relu	Sigmoid	50	32	0.9846	0.9768
3	64	32	3	2	Relu	Sigmoid	50	32	0.9898	0.9834
5	64	32	3	2	Relu	Sigmoid	50	32	0.9956	0.9834
10	64	32	3	2	Relu	Sigmoid	50	32	0.9982	0.9856

4.6. Comparison of Performance with Alternative Datasets

In analyzing the consolidated performance of various models through four datasets, distinct patterns emerge that offer insights into model suitability for different contexts. Neural-network-based models, particularly CNN-BiGRU and GRU-based architectures, consistently exhibit high accuracy and F1 scores through all datasets. For example, CNN-BiGRU achieves a notably high accuracy of 98.54% on Dataset 1 and 98.21% on Dataset 4, demonstrating adaptability through varied data distributions (see Table 11). This suggests that CNN-BiGRU, with its hybrid deep learning architecture, is well suited for capturing complex patterns and dependencies in diverse datasets, outperforming simpler models such as Decision Trees and SVM, which show significantly lower performance, particularly on Dataset 2.

Table 11. Consolidated performance comparison of models across four datasets.

Algorithm	Dataset 1		Dataset 2		Dataset 3		Dataset 4	
	Accuracy	F1 Score	Accuracy	F1 Score	Accuracy	F1 Score	Accuracy	F1 Score
Decision Tree	0.9747	0.9758	0.8482	0.8726	0.9564	0.9576	0.9245	0.9253
Random Forest	0.9731	0.9738	0.7386	0.7448	0.9564	0.9561	0.8411	0.8275
K-Nearest Neighbors	0.9309	0.9353	0.6941	0.7449	0.8914	0.8981	0.7855	0.7899
SVM	0.8871	0.8867	0.6769	0.7728	0.8372	0.8412	0.7182	0.6945
AdaBoost	0.9209	0.9235	0.8126	0.8570	0.9037	0.9055	0.8080	0.8254
LSTM	0.9708	0.9716	0.9638	0.9692	0.9648	0.9656	0.9768	0.9767
BiLSTM	0.9724	0.9729	0.9605	0.9663	0.9717	0.9720	0.9763	0.9764
GRU	0.9754	0.9760	0.9627	0.9683	0.9694	0.9697	0.9738	0.9736
BiGRU	0.9708	0.9709	0.9633	0.9686	0.9648	0.9647	0.9763	0.9760
CNN-LSTM	0.9724	0.9728	0.9583	0.9645	0.9702	0.9707	0.9724	0.9720
CNN-BiLSTM	0.9731	0.9739	0.9588	0.9649	0.9732	0.9737	0.9721	0.9720
CNN-GRU	0.9693	0.9703	0.9583	0.9643	0.9763	0.9764	0.9721	0.9725
CNN-BiGRU	0.9854	0.9856	0.9722	0.9593	0.9817	0.9715	0.9821	0.9725

Traditional machine learning algorithms like Decision Trees, Random Forests, and SVM show a clear performance drop, especially on Dataset 2, where Decision Tree accuracy falls to 84.82% and SVM accuracy is the lowest through all models, at 67.69%. This disparity highlights the limitations of these algorithms in handling certain datasets, possibly due to the lack of feature extraction capabilities present in deep learning models. Random Forest and K-Nearest Neighbors, while outperforming SVM, still struggle on Dataset 2, suggesting that they may be better suited for datasets with simpler structures or when interpretability is prioritized over accuracy. This performance contrast presents the effectiveness of deep

learning approaches, especially hybrid models like CNN-BiGRU, in delivering consistently high results through varied dataset characteristics. These findings point to the importance of model selection based on dataset complexity and the trade-offs between interpretability and predictive power.

The three datasets used—Dataset 1, Dataset 2, and Dataset 3—were sourced from various environments with distinct vocabulary, linguistic patterns, and demographic influences. Dataset 1 contains open-source SMS data with common linguistic patterns, while Dataset 2, sourced from Hugging Face, features a broader array of spam scenarios, enhancing the coverage of varied phishing techniques. Dataset 3 initially included three categories (ham, smishing, and spam), but we reorganized it into binary classes (“ham” and “smishing”) for consistency. Each dataset also exhibits a unique balance of legitimate (ham) to phishing (smishing) messages: for example, Dataset 2 contains a higher proportion of spam, while Dataset 3 originally encompassed both spam and smishing content. By training on these varied distributions, we assessed our model’s ability to maintain performance through different class ratios, offering insights into how effectively it can detect phishing attempts irrespective of class imbalances.

To further enhance generalizability, we created a fourth combined dataset by merging all three primary datasets. This aggregated dataset provides a greater diversity in message characteristics, improving model robustness by exposing it to a wider variety of phishing and legitimate messages. Performance gains observed on this combined dataset highlight the benefit of training on a comprehensive dataset, allowing the model to better capture a full range of phishing patterns, enhancing its applicability in various messaging environments.

As shown in Table 11, model performance varies through individual datasets, reflecting their unique compositions. Models generally performed best on Dataset 1, which has a relatively straightforward structure, whereas Dataset 2, with more varied spam scenarios, presented greater challenges and led to moderate accuracy. Dataset 3 performed comparably to Dataset 1, while Dataset 4 demonstrated the highest overall accuracy and F1 scores. These findings underscore the importance of a consolidated dataset that encompasses varied message types for developing models capable of generalizing effectively through unseen data sources.

4.7. Exploring the Explainable AI Method

In the context of machine learning models, interpretability is crucial, particularly in applications such as cybersecurity, where understanding model predictions can provide insights into underlying threats. To address this need, we employed the Local Interpretable Model-agnostic Explanation (LIME) to elucidate the decision-making process of our hybrid deep learning model used for smishing attack detection.

The LIME is an interpretability framework that offers insight into individual predictions by approximating the complex model with a simpler, interpretable model in the vicinity of a specific prediction. By perturbing the input data and observing the changes in the output, LIME generates local approximations that highlight which features contribute significantly to the final prediction. This process enhances the transparency of our model and facilitates a deeper understanding of how specific textual features influence smishing detection.

Mathematical Foundation:

Given a trained model $f : \mathbb{R}^n \rightarrow \mathbb{R}$ that maps input features $x \in \mathbb{R}^n$ to a predicted outcome, the LIME operates as follows:

1. Local Neighborhood Creation: For a given instance x_0 for which we wish to obtain an explanation, the LIME generates a dataset of perturbed samples $\{(x_i, f(x_i))\}_{i=1}^m$. These perturbed samples x_i are created by randomly modifying the original instance x_0 in a controlled manner. The perturbation function is often defined as

$$\tilde{x}_i \sim \text{Perturb}(x_0)$$

The goal is to create a set of samples around x_0 that retains its local context.

2. Weighted Sampling: The LIME assigns a weight w_i to each perturbed sample based on its proximity to x_0 . This weighting can be formulated using a kernel function $K(x_0, x_i)$, typically chosen as an exponential decay function:

$$w_i = K(x_0, x_i) = \exp\left(-\frac{d(x_0, x_i)^2}{\sigma^2}\right)$$

where $d(x_0, x_i)$ is a distance metric (often Euclidean) between the original instance and the perturbed sample and σ is a bandwidth parameter controlling the size of the neighborhood.

3. Fitting an Interpretable Model: With the perturbed samples and their corresponding predictions $f(x_i)$ and weights w_i , the LIME fits a simple, interpretable model $g : \mathbb{R}^n \rightarrow \mathbb{R}$ (commonly a linear model) that minimizes the following objective function:

$$\hat{g} = \arg \min_g \sum_{i=1}^m w_i (g(x_i) - f(x_i))^2 + \lambda \Omega(g)$$

where $\Omega(g)$ is a complexity penalty term that discourages overfitting and λ is a regularization parameter.

4. Interpretation of the Explanation: The resulting interpretable model g provides coefficients that indicate the contribution of each feature to the prediction made by the complex model f for the instance x_0 . This is typically represented as

$$g(x) = \sum_{j=1}^n \theta_j x_j$$

where θ_j represents the importance of feature j . The sign and magnitude of these coefficients allow us to infer which features are driving the model's prediction, thus providing an explanation that is both human-readable and mathematically grounded.

In our implementation, we first trained the proposed model on a dataset of SMS messages, where each message is labeled as either benign or a potential smishing attempt. After training, we selected specific instances of interest from the test set to apply LIME. By using LIME, we generated explanations for these instances, which involved analyzing the impact of individual features (e.g., specific words or phrases) on the model's predictions.

The LIME results revealed which textual features were most influential in classifying messages as smishing (see Table 12 and Figure 3). For instance, words commonly associated with phishing attempts, such as 'urgent', 'free', or 'click here', emerged as significant indicators.

Table 12. Feature importance table for dataset 3.

Index	Feature	Importance
1	call	0.3059
2	txt	0.1122
3	www	0.0717
4	http	0.0492
5	me	0.0368
...	...	...
4996	greet	0.0000
4997	green	0.0000
4998	great	0.0000
4999	gravity	0.0000
5000	ã¼	0.0000

Figure 4 presented here offers a LIME analysis for a sample text classified by a model developed for detecting smishing attacks. The sample text provided is a typical example of

a smishing message, using sms like ‘Try CHIT-CHAT on your mobile now!’ and including details that create urgency or curiosity, which are common tactics in smishing attempts.

The words ‘mobile’ and ‘16’ are highlighted as the most influential in classifying this message as smishing, with positive contributions to the ‘smish’ prediction. The LIME provides an interpretability layer by identifying these specific words as critical features that influenced the model’s decision. Words like ‘mobile’ likely contributed due to their association with mobile device-targeted scams, and ‘16’ might be linked to age-related or verification prompts often seen in phishing attempts.

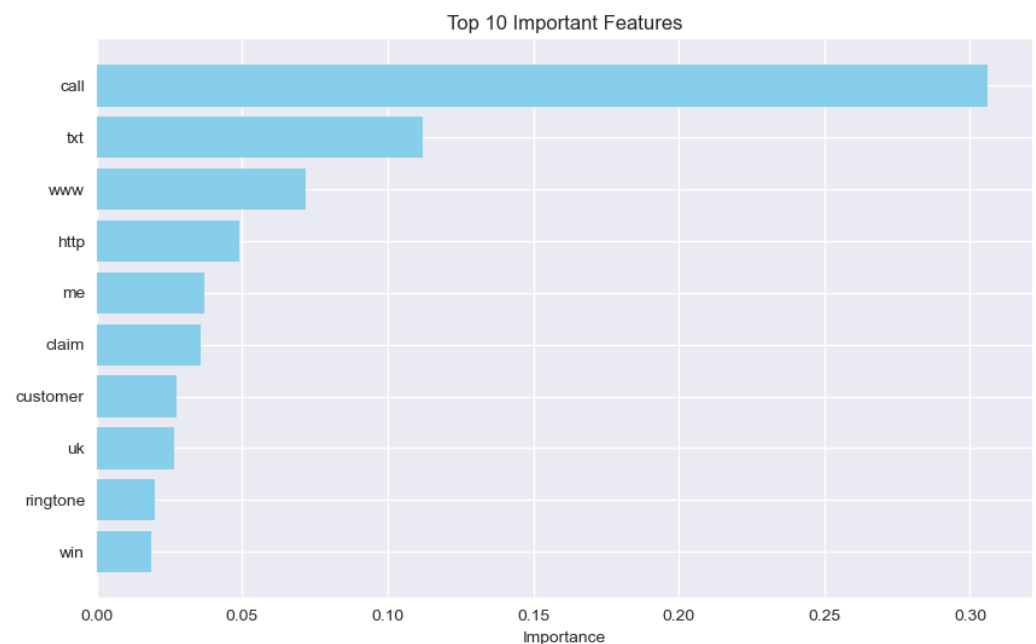
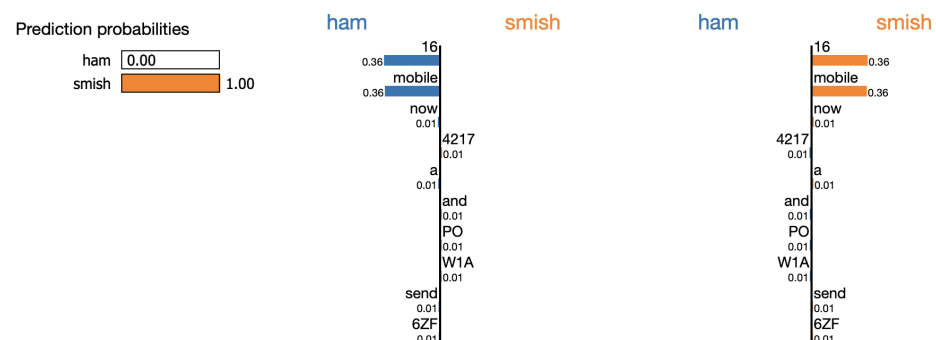


Figure 3. Pictorial representations of the top ten important features on dataset 3.

Explanation for sample: 'Wanna have a laugh? Try CHIT-CHAT on your mobile now! Logon by txtng the word: CHAT and send it to No: 8883 CM PO Box 4217 London W1A 6ZF 16+ 118p/msg rcvd'



Text with highlighted words

Wanna have a laugh? Try CHIT-CHAT on your **mobile** now! Logon by txtng the word: CHAT and send it to No: 8883 CM PO Box 4217 London W1A 6ZF **16**+ 118p/msg rcvd

LIME Feature Importance Table:

Feature	Importance
0 16	0.364695
1 mobile	0.359041
2 now	0.014695
3 4217	-0.013392
4 a	0.012257
5 and	-0.011476
6 PO	-0.011394
7 W1A	-0.010949
8 send	0.009659
9 6ZF	0.008882

Figure 4. LIME analysis for a sample text classified by a proposed model-1.

The LIME feature importance table lists the top words influencing the prediction, where '16' and 'mobile' have the highest positive weights. This indicates that the model detects such keywords as strong indicators of a smishing attack. Negative weights for words like 'W1A' or 'PO' suggest these terms do not contribute positively to the smishing classification, possibly due to their lower correlation with typical smishing language or patterns.

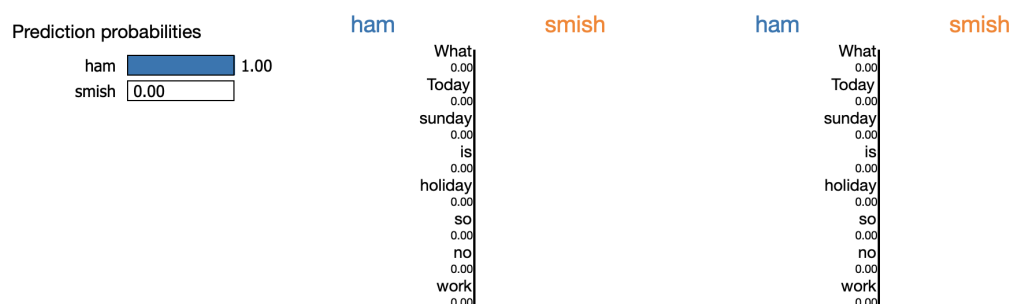
The plot shows the prediction probabilities, where the model assigns a high probability of 'smish' (1.00), correctly identifying the message as smishing. This suggests that the model is effective in recognizing this type of attack, with minimal confusion between 'ham' and 'smish'.

Similarly, the model has a very high confidence in classifying this message as 'ham', with a probability of 1.00 and a 'smish', with a probability of 0.00. This result reflects the model's ability to correctly identify non-threatening or casual messages as non-smishing.

The words in the sentence (see Figure 5), such as 'What', 'Today', 'Sunday', 'holiday', and 'no work', have no significant feature importance values contributing to the classification as 'ham' or 'smish'. This absence of influential terms indicates that these words do not match any typical smishing patterns or keywords that the model has learned to associate with smishing attacks.

By showing no significant word contributions for either classification, the LIME explanation reinforces the model's reliability. The model correctly identifies this message as 'ham' without falsely attributing any suspicious characteristics to neutral words. In the context of the study, this example demonstrates how the model is not only effective in identifying smishing attacks but also resilient in not over-flagging benign messages, which is crucial for reducing false positives in real-world applications.

Explanation for sample: 'What Today-sunday..sunday is holiday..so no work..'



Text with highlighted words

What Today-sunday..sunday is holiday..so no work..

LIME Feature Importance Table:

Feature	Importance
0 What	0.0
1 Today	0.0
2 sunday	0.0
3 is	0.0
4 holiday	0.0
5 so	0.0
6 no	0.0
7 work	0.0

Figure 5. LIME analysis for a sample text classified by a proposed model-2.

4.8. Computational Efficiency and Resource Requirements

This section provides a comprehensive overview of the computational efficiency and resource demands of various models, crucial for evaluating their feasibility in resource-constrained settings like smishing detection. Table 13 reveals that training times vary widely, with bidirectional models like BiLSTM and CNN-BiLSTM requiring significantly longer times per epoch due to their ability to capture both past and future context. In contrast, models like GRU and CNN-BiGRU demonstrate lower training times, suggesting they may be more computationally efficient while still providing sufficient contextual

understanding for the task. Testing times, however, are consistently short through all models, ranging from 1 to 2 s, making each option feasible for real-time applications where quick responses are critical.

Table 14 shows similar memory requirements through models, all around 3 MB, indicating that they are practical for deployment even on devices with limited storage. Models like CNN-BiGRU and CNN-GRU strike a favorable balance with lower parameter counts and smaller memory footprints, making them optimal choices for efficient yet powerful smishing detection in edge environments.

Table 13. Training and testing times for different models.

Model	Average Training Time per Epoch	Total Training Time	Total Testing Time
LSTM	21.5 s	Approximately 17.92 min (1075 s)	Approximately 2 s
BiLSTM	191.4 s	9571 s	2 s
GRU	11.5 s	Approximately 575 s	1 s
BiGRU	15.3 s	765 s	2 s
CNN-LSTM	37.82 s	1891 s	2 s
CNN-BiLSTM	206.92 s	10346 s	2 s
CNN-GRU	19.74 s	987 s	2 s
CNN-BiGRU	8 s	400 s	1 s

Table 14. Model parameters and memory size.

Model	Parameters	Approximate Memory Size (MB)
LSTM	790,057	3.0138
BiLSTM	807,113	3.0789
GRU	785,897	2.9980
BiGRU	798,793	3.0472
CNN-LSTM	785,785	2.9975
CNN-BiLSTM	788,937	3.0096
CNN-GRU	785,049	2.9947
CNN-BiGRU	787,465	3.0039

4.9. Comparison with Prior Studies

The effectiveness of five different SMS spam detection techniques was examined in this comparison research. DSmishSMS, Hybrid, SMS Spam Filter, Smishing Detector, and Spam Transformer are the approaches in Table 15. Our investigation also included a proposed method. DSmishSMS employed a machine learning strategy along with the T.A. Almeida Collected Dataset and the Pinterest.com text SMS dataset. Using the Random Forest classification technique, 97.93% accuracy was attained. Hybrid utilized deep learning and the T.A. Almeida dataset. It used the CNN-LSTM classification technique and attained 98.37% accuracy. SMS Spam Filter incorporated a deep learning strategy using the UCI Benchmark dataset. It also employed the CNN-LSTM classification technique and obtained 99.44% accuracy. Smishing Detector utilized machine learning and the SMS Spam Collection dataset from UCI. It used the Naive Bayes classification technique and attained 96.12% accuracy. Spam Transformer used a deep learning strategy and two datasets, SMS Spam Collection and UkMI's Twitter. Throughout the analysis, a 98.92% accuracy score was reported. The proposed method incorporated a deep learning strategy and the T.A. Almeida dataset. It utilized the CNN-Bi-GRU classification algorithm and attained an accuracy of 99.82%.

Table 15. Analysis of our proposed system versus prior research.

Study	Approach Used	Dataset Used	Classification	Accuracy
SMS Spam Filter [23]	Deep Learning	UCI Benchmark	CNN-LSTM	99.44%
Hybrid [24]	Deep Learning	T.A Almeida	CNN-LSTM	98.37%
Smishing Detector [10]	Machine- Learning	UCI's SMS Spam Collection	Naïve Bayes	96.12%
DSmishSMS [27]	Machine-Learning	T.A Almeida Collected Dataset and Pinterest.com text SMS	Random Forest	97.93%
Spam Transformer [28]	Deep Learning	SMS Spam Collection and UtkMI's Twitter	CNN-LSTM	98.92%
Phishing Detection [31]	Deep Learning	Phishing emails (Collection and Nazario Phishing Corpus The Enron dataset)	BiLSTM (URL, No_URL)	97.9%
Proposed Method	Deep Learning	Smishing Collection [32]	CNN-Bi-GRU	99.82%

In comparing the detection performance through deep learning models, our study demonstrates significant advantages in accuracy and dataset diversity by incorporating four distinct SMS datasets. In contrast to Baardsen et al. [31], who utilized email datasets—one containing URLs (Phishing emails and Nazario Phishing Corpus) and another without URLs (Enron dataset)—our best-performing model, CNN-BiGRU, achieved an impressive accuracy of 99.82%, surpassing Baardsen et al.'s highest results of 97.9% for URL data and 96.8% for non-URL data with models like BiLSTM, BiGRU, and CNN. Other high-performing models in our study, such as CNN-GRU (96.93%), CNN-BiLSTM (97.31%), and CNN-LSTM (99.44%), also showcased consistently high accuracy through multiple datasets.

Overall, our analysis demonstrates that the proposed method employing a deep learning approach and the CNN-Bi-GRU classification algorithm is the most successful way to detect SMS phishing, with an accuracy rating of 99.82%. With an accuracy score of 99.44%, the SMS Spam Filter based on deep learning and the CNN-LSTM classification algorithm also demonstrated strong performance.

5. Conclusions and Future Work

This study explored the application of hybrid deep learning models, specifically the CNN-Bi-GRU architecture, for detecting SMS phishing attacks. The proposed model demonstrated superior performance in terms of accuracy, precision, recall, and F1-score, significantly outperforming traditional classifiers such as KNN, decision tree, random forest, SVM, AdaBoost, LSTM, and GRU. With an accuracy of 99.82% and a high F1-score of 98.56%, the CNN-Bi-GRU model proved to be an effective tool for enhancing cybersecurity in the context of SMS phishing detection.

The results present the potential of deep learning models in fortifying SMS phishing security protocols, offering a robust solution for accurately identifying and classifying phishing attempts. By leveraging the strengths of both CNNs and Bi-GRUs, the model efficiently captured and processed the complex patterns within the SMS Phishing Collection dataset, making it a valuable asset in the fight against cybercrime.

In future work, the focus will be on expanding the dataset to enhance the model's performances and exploring alternative deep learning architectures, such as attention mechanisms and transformers, to further improve detection accuracy. Additionally, efforts

will be made to optimize the model for real-time SMS phishing detection on mobile devices, ensuring both efficiency and speed. Addressing more complex and sophisticated phishing attacks will also be a priority, as well as exploring cross-domain learning to create a unified detection framework through different phishing methods. Finally, integrating these insights into user education and awareness initiatives could further reduce the effectiveness of SMS phishing attacks, contributing to a more secure digital environment.

Author Contributions: Conceptualization, T.M., M.A.H.P. and M.H.A.; methodology, T.M., M.A.H.P. and M.H.A.; validation, T.M., M.A.H.P. M.H.A., M.S.H. and K.A.; formal analysis, T.M., M.S.H. and K.A.; investigation, T.M., M.S.H. and K.A.; resources, T.M., M.S.H. and K.A.; data curation, T.M., M.S.H. and K.A.; writing—original draft preparation, T.M., M.A.H.P. and M.H.A.; writing—review and editing, T.M., M.S.H. and K.A.; visualization, T.M., M.A.H.P. and M.H.A.; supervision, T.M., M.S.H. and K.A. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Informed Consent Statement: Informed consent was obtained from all subjects involved in the study.

Data Availability Statement: The data used to support the findings of this study are available upon reasonable request to the corresponding author.

Acknowledgments: We acknowledge the use of AI tools for enhancing the quality of the paper, particularly for grammar checking.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

SMS	Short Message Service
CNN	Convolutional Neural Networks
Bi-GRU	Bidirectional Gated Recurrent Units
ML	Machine Learning
DL	Deep Learning (ML)
RNN	Recurrent neural network
XAI	Explainable AI

References

1. Cost of a Data Breach Report 2021. Available online: <https://www.ibm.com/security/data-breach> (accessed on 11 August 2024).
2. Difference Between Spam and Phishing Mail. Available online: <https://www.tutorialspoint.com/difference-between-spam-and-phishing-mail> (accessed on 11 August 2024).
3. Datta, N.; Mahmud, T.; Aziz, M.T.; Das, R.K.; Hossain, M.S.; Andersson, K. Emerging Trends and Challenges in Cybersecurity Data Science: A State-of-the-Art Review. In Proceedings of the 2024 Parul International Conference on Engineering and Technology (PICET), Vadodara, India, 3–4 May 2024; pp. 1–7.
4. 6 Reasons Why SMS Is More Effective than Email Marketing—CallHub. 2016. Available online: <https://callhub.io/6-reasons-sms-effectiveemail-marketing/> (accessed on 11 April 2024).
5. Khan, F.; Mustafa, R.; Tasnim, F.; Mahmud, T.; Hossain, M.S.; Andersson, K. Exploring BERT and ELMo for Bangla Spam SMS Dataset Creation and Detection. In Proceedings of the 2023 26th International Conference on Computer and Information Technology (ICCIT), Cox’s Bazar, Bangladesh, 13–15 December 2023; IEEE: Piscataway, NJ, USA, 2023; pp. 1–6.
6. Ayeni, R.K.; Adebiyi, A.A.; Okesola, J.O.; Igbekele, E. Phishing Attacks and Detection Techniques: A Systematic Review. In Proceedings of the 2024 International Conference on Science, Engineering and Business for Driving Sustainable Development Goals (SEB4SDG), Omu-Aran, Nigeria, 2–4 April 2024; IEEE: Piscataway, NJ, USA, 2024; pp. 1–17.
7. Ali, M.M.; Mohd Zaharon, N.F. Phishing—A cyber fraud: The types, implications and governance. *Int. J. Educ. Reform* **2024**, *33*, 101–121. [CrossRef]
8. Nadeem, M.; Zahra, S.W.; Abbasi, M.N.; Arshad, A.; Riaz, S.; Ahmed, W. Phishing attack, its detections and prevention techniques. *Int. J. Wirel. Secur. Netw.* **2023**, *1*, 13–25.
9. Jakobsson, M. Two-factor inauthentication—The rise in SMS phishing attacks. *Comput. Fraud. Secur.* **2018**, *2018*, 6–8. [CrossRef]
10. Mishra, S.; Soni, D. Smishing Detector: A security model to detect smishing through SMS content analysis and URL behavior analysis. *Future Gener. Comput. Syst.* **2020**, *108*, 803–815. [CrossRef]

11. What Is Phishing | Attack Techniques & Scam Examples. Learning Center. Available online: <https://www.imperva.com/learn/application-security/phishing-attack-scam/> (accessed on 11 May 2024).
12. Phishing for Information: Spearphishing Link, Sub-Technique T1598.003—Enterprise | MITRE ATT&CK®. Available online: <https://attack.mitre.org/techniques/T1598/003/> (accessed on 11 May 2024).
13. 2022 Data Breach Investigations Report. Available online: <https://www.verizon.com/business/en-gb/resources/reports/dbir/> (accessed on 11 August 2024).
14. Internet Crime Complaint Center (IC3) Releases 2020 Internet Crime Report, Including COVID-19 Scam Statistics. Available online: <https://www.ic3.gov/Media/News/2021/210325.aspx> (accessed on 11 August 2024).
15. Increasing Cybercrime: UN Reports 350 Percent Rise in Phishing Websites During Pandemic. 2020. Available online: <https://www.newindianexpress.com/business/2020/aug/08/increasing-cybercrime-un-reports-350-per-cent-rise-in-phishing-websites-during-pandemic-2180777.html> (accessed on 11 August 2024).
16. Mahmud, T.; Ptaszynski, M.; Masui, F. Deep Learning Hybrid Models for Multilingual Cyberbullying Detection: Insights from Bangla and Chittagonian Languages. In Proceedings of the 2023 26th International Conference on Computer and Information Technology (ICCIT), Cox's Bazar, Bangladesh, 13–15 December 2023; IEEE: Piscataway, NJ, USA, 2023; pp. 1–6.
17. Mahmud, T.; Ptaszynski, M.; Masui, F. Automatic Vulgar Word Extraction Method with Application to Vulgar Remark Detection in Chittagonian Dialect of Bangla. *Appl. Sci.* **2023**, *13*, 11875. [\[CrossRef\]](#)
18. Mahmud, T.; Ptaszynski, M.; Masui, F. Exhaustive Study into Machine Learning and Deep Learning Methods for Multilingual Cyberbullying Detection in Bangla and Chittagonian Texts. *Electronics* **2024**, *13*, 1677. [\[CrossRef\]](#)
19. Almeida, T.A.; Hidalgo, J.M.G.; Yamakami, A. Contributions to the study of SMS spam filtering: New collection and results. In Proceedings of the 11th ACM Symposium on Document Engineering, Mountain View, CA, USA, 19–22 September 2011; pp. 259–262.
20. Naher, S.R.; Sultana, S.; Mahmud, T.; Aziz, M.T.; Hossain, M.S.; Andersson, K. Exploring Deep Learning for Chittagonian Slang Detection in Social Media Texts. In Proceedings of the 2024 International Conference on Electrical, Computer and Energy Technologies (ICECET), Sydney, Australia, 25–27 July 2024; IEEE: Piscataway, NJ, USA, 2024; pp. 1–6.
21. Joo, J.W.; Moon, S.Y.; Singh, S.; Park, J.H. S-Detector: An enhanced security model for detecting Smishing attack for mobile computing. *Telecommun. Syst.* **2017**, *66*, 29–38. [\[CrossRef\]](#)
22. Sonowal, G. Detecting phishing SMS based on multiple correlation algorithms. *SN Comput. Sci.* **2020**, *1*, 361. [\[CrossRef\]](#) [\[PubMed\]](#)
23. Roy, P.K.; Singh, J.P.; Banerjee, S. Deep learning to filter SMS Spam. *Future Gener. Comput. Syst.* **2020**, *102*, 524–533. [\[CrossRef\]](#)
24. Ghourabi, A.; Mahmood, M.A.; Alzubi, Q.M. A hybrid CNN-LSTM model for SMS spam detection in arabic and english messages. *Future Internet* **2020**, *12*, 156. [\[CrossRef\]](#)
25. Jain, A.K.; Yadav, S.K.; Choudhary, N. A novel approach to detect spam and smishing SMS using machine learning techniques. *Int. J. E-Serv. Mob. Appl.* **2020**, *12*, 21–38. [\[CrossRef\]](#)
26. Xia, T.; Chen, X. A discrete hidden Markov model for SMS spam detection. *Appl. Sci.* **2020**, *10*, 5011. [\[CrossRef\]](#)
27. Mishra, S.; Soni, D. DSmsSMS—A System to Detect Smishing SMS. *Neural Comput. Appl.* **2023**, *35*, 4975–4992. [\[CrossRef\]](#) [\[PubMed\]](#)
28. Liu, X.; Lu, H.; Nayak, A. A spam transformer model for SMS spam detection. *IEEE Access* **2021**, *9*, 80253–80263. [\[CrossRef\]](#)
29. Mishra, S.; Soni, D. Implementation of ‘smishing detector’: An efficient model for smishing detection using neural network. *SN Comput. Sci.* **2022**, *3*, 189. [\[CrossRef\]](#)
30. Mambina, I.S.; Ndiwile, J.D.; Michael, K.F. Classifying Swahili Smishing Attacks for Mobile Money Users: A Machine-Learning Approach. *IEEE Access* **2022**, *10*, 83061–83074. [\[CrossRef\]](#)
31. Baardsen, A. Phishing and Social Engineering Attack Detection by Applying Intention Detection Methods. Master's Thesis, NTNU, Trondheim, Norway, 2022.
32. SMS Smishing Collection Data Set. Kaggle. Available online: <https://www.kaggle.com/datasets/galactus007/sms-smishing-collection-data-set> (accessed on 11 December 2023).
33. Mishra, S.; Soni, D. Sms phishing dataset for machine learning and pattern recognition. In Proceedings of the International Conference on Soft Computing and Pattern Recognition, Seattle, WA, USA, 14–16 December 2022; Springer: Cham, Switzerland, 2022; pp. 597–604.
34. Mahmud, T.; Ptaszynski, M.; Eronen, J.; Masui, F. Cyberbullying detection for low-resource languages and dialects: Review of the state of the art. *Inf. Process. Manag.* **2023**, *60*, 103454. [\[CrossRef\]](#)
35. Mahmud, T.; Karim, R.; Chakma, R.; Chowdhury, T.; Hossain, M.S.; Andersson, K. A Benchmark Dataset for Cricket Sentiment Analysis in Bangla Social Media Text. *Procedia Comput. Sci.* **2024**, *238*, 377–384. [\[CrossRef\]](#)
36. Akter, T.; Akter, M.S.; Mahmud, T.; Islam, D.; Hossain, M.S.; Andersson, K. Evaluating Machine Learning Methods for Bangla Text Emotion Analysis. In Proceedings of the 2024 Asia Pacific Conference on Innovation in Technology (APCIT), Mysore, India, 26–27 July 2024; IEEE: Piscataway, NJ, USA, 2024; pp. 1–6.
37. Mahmud, T.; Akter, T.; Aziz, M.T.; Uddin, M.K.; Hossain, M.S.; Andersson, K. Integration of NLP and Deep Learning for Automated Fake News Detection. In Proceedings of the 2024 Second International Conference on Inventive Computing and Informatics (ICICI), Bangalore, India, 11–12 June 2024; IEEE: Piscataway, NJ, USA, 2024; pp. 398–404.

38. Bappy, A.D.; Mahmud, T.; Kaiser, M.S.; Shahadat Hossain, M.; Andersson, K. A BERT-Based Chatbot to Support Cancer Treatment Follow-Up. In Proceedings of the International Conference on Applied Intelligence and Informatics, Dubai, United Arab Emirates, 29–31 October 2023; Springer: Cham, Switzerland, 2023; pp. 47–64.
39. Rahman, M.A.; Begum, M.; Mahmud, T.; Hossain, M.S.; Andersson, K. Analyzing Sentiments in eLearning: A Comparative Study of Bangla and Romanized Bangla Text using Transformers. *IEEE Access* **2024**, *12*, 89144–89162. [[CrossRef](#)]
40. Rayhanuzzaman; Mahmud, T.; Das, U.K.; Naher, S.R.; Hossain, M.S.; Andersson, K. Investigating the Effectiveness of Deep Learning and Machine Learning for Bangla Poems Genre Classification. In Proceedings of the 2023 4th International Conference on Intelligent Technologies (CONIT), Bangalore, India, 21–23 June 2024; IEEE: Piscataway, NJ, USA, 2024; pp. 1–6.
41. Habiba, S.U.; Mahmud, T.; Naher, S.R.; Aziz, M.T.; Rahman, T.; Datta, N.; Hossain, M.S.; Andersson, K.; Kaiser, M.S. Deep Learning Solutions for Detecting Bangla Fake News: A CNN-Based Approach. In Proceedings of the Trends in Electronics and Health Informatics: TEHI 2023, Dhaka, Bangladesh, 20–21 December 2023; p. 107.
42. Barman, S.; Biswas, M.R.; Marjan, S.; Nahar, N.; Imam, M.H.; Mahmud, T.; Kaiser, M.S.; Hossain, M.S.; Andersson, K. A Two-Stage Stacking Ensemble Learning for Employee Attrition Prediction. In Proceedings of the International Conference on Trends in Electronics and Health Informatics, Dhaka, Bangladesh, 20–21 December 2023; Springer: Singapore, 2023; pp. 119–132.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.